

Huffman Coding Algorithm

What is Encoding ?

Encoding, in computers, can be defined as the process of transmitting or storing sequence of characters efficiently.

Every information in computer science is **encoded** as strings of **1s and 0s**.

The objective of information theory is to usually transmit information using fewest number of bits in such a way that every encoding is unambiguous

There are two types of encoding schemes

Fixed-Length encoding –

Every character is assigned a binary code using same number of bits.

Assuming that each character uses 8 bits.

Thus, a string like “**aabacdad**” can require 64 bits (8 bytes) for storage or transmission.

Variable- Length encoding –

this scheme uses variable number of bits for encoding the characters depending on their frequency in the given text.

Variable- Length encoding

Ex:

for a given string like “**aabacdad**”, frequency of characters ‘a’, ‘b’, ‘c’ and ‘d’ is 4,1,1 and 2 respectively.

Since ‘a’ occurs more frequently than ‘b’, ‘c’ and ‘d’, it uses least number of bits, followed by ‘d’, ‘b’ and ‘c’.

Lets randomly assign binary codes to each character as follows-

- **a 0**
- b 011**
- c 111**
- d 11**

Thus, the string “aabacdad” gets encoded to **00011011111011**

(0 | 0 | 011 | 0 | 111 | 11 | 0 | 11)

using fewer number of bits compared to fixed-length encoding scheme. (i.e. 14 vs 64)

Problem with this strategy

a 0
b 011
c 111
d 11

the real problem lies with the decoding phase.

If we try and decode the string **00011011111011**, it will be quite ambiguous since, it can be decoded to the multiple strings,

few of which are-

- **aaadacdad** (0 | 0 | 0 | 11 | 0 | 111 | 11 | 0 | 11)
aaadbcbad (0 | 0 | 0 | 11 | 011 | 111 | 0 | 11)
aabbcb (0 | 0 | 011 | 011 | 111 | 011) ... and so on

Prefix rule (To prevent such ambiguities during decoding)

Encoding phase should satisfy the “**prefix rule**”

which states that no binary code should be a prefix of another code.

(i.e. 0, is a prefix of binary code for b i.e 011, is “non-prefix”)

This will produce uniquely **decodable codes**.

Ex: Applying prefix Rule for encoding

Lets reconsider assigning the binary codes to characters 'a', 'b', 'c', 'd'.

a 0

b 11

c 101

d 100

Using the above codes, string **“aabacdad”** gets encoded to

001101011000100 (0 | 0 | 11 | 0 | 101 | 100 | 0 | 100).

Now, we can decode it back to string **“aabacdad”**.

Huffman Encoding-

Developed by **David Huffman** in 1951, Huffman Encoding method is used for finding efficient coding,

- This technique is the basis for all data compression and encoding schemes.
- It uses variable-length encoding scheme for assigning binary codes to characters depending on how frequently they occur in the given text

Huffman Encoding-

Algorithm for creating the Huffman Tree-

Step 1- Create a leaf node for each character and build a min heap using all the nodes (the frequency value is used to compare two nodes in min heap)

Step 2- Repeat Steps 3 to 5 while heap has more than one node

Step 3- Extract two nodes, say x and y, with minimum frequency from the heap

Step 4- Create a new internal node z with x as its left child and y as its right child.
 $\text{frequency}(z) = \text{frequency}(x) + \text{frequency}(y)$

Step 5- Add z to min heap

Step 6- Last node in the heap is the root of Huffman tree

Ex

for a given string like “**aabacdad**”, frequency of characters ‘a’, ‘b’, ‘c’ and ‘d’ is 4,1,1 and 2 respectively.

- Arrange the characters in ascending order of their frequency and apply Huffman algorithm
- Huffman tree will generate the following code
- 0 gets decoded to ‘a’
- 110 gets decoded to ‘b’
- 111 gets decoded to ‘c’
- 10 gets decoded to ‘d’

Analysis

“aabacdad”,

0 gets decoded to 'a' (4)
110 gets decoded to 'b' (1)
111 gets decoded to 'c' (1)
10 gets decoded to 'd' (2)

Default encoding

Here decoding is not required as ASCII representation is universally known and unique

=> Number of bits used for encoding = $8 * 8 = 64$ bits

Uniform encoding (with less number of bits)

total bits required = decoding scheme + encoded message

decoding scheme => [Ascii bits for 4 characters + 2 bit representation for 4 characters]

encoded message => [2 bits for each character in the string]

=> $(8*4 + 4*2) + (8*2) = (32 + 8) + 16 = 56$ bits

Variable encoding = decoding scheme + encoded message

=> total bits required = decoding scheme + encoded message

=> [Ascii bits for 4 characters + bit representation for 4 characters]

=> encoded message => [a*4+b*1+c*1+d*2 bits (for characters used in the string)]

=> $(8*4 + 9) + (1*4 + 3*1 + 3*1 + 2*2) = (32 + 9) + (14) = 55$ bits

=> The real benefit is realized with larger size strings

Compare encoding of
aaaabadd
using
Uniform encoding and Variable encoding methods

- ☐ Find the bits used in Uniform encoding ?
- ☐ Find the bits used in Variable encoding (Huffman encoding) ?

Compare encoding of
aaaabadd

using

Uniform encoding and Variable encoding methods

0 gets decoded to 'a' (5)
10 gets decoded to 'b' (1)
11 gets decoded to 'd' (3)

- ☐ Find the bits used in Uniform encoding ?
- ☐ Find the bits used in Variable encoding (Huffman encoding) ?
- ☐ Uniform Encoding = $3*8 + 3*2 + 9*2 = 24 + 6 + 18 = 48$ bits
- ☐ Variable Encoding = $3*8 + 7 + (5*1 + 1*2 + 3*2) = 24 + 7 + 13 = 44$ bits

A file containing 6 unique characters and frequency of each character is given:

c=34

d=9

g=35

u=2

m=2

a=100

How many bits are required to store this file using Huffman Encoding?

A file containing 6 unique characters and frequency of each character is given:

c=34 - **110 / 011**
d=9 - **1110 / 0101**
g=35 - **10 / 00**
u=2 - **11110 / 01000**
m=2 - **11111 / 01001**
a=100 - **0 / 1**

Number of bits required to store this file using Huffman Encoding?

Size of file = 182

Bits for representing $8*6 + 20 = 68$

Bits for encoding = $34*3 + 9*4 + 35*2 + 2*5 + 2*5 + 100*1$
 $(102 + 36 + 70 + 10 + 10 + 100) = 328$

Total bits = $328 + 68 = 396$

Consider a file contents are encoded as **"0101101101001"**
using the following Huffman encoding

c=34	- 011
d=9	- 0101
g=35	- 00
u=2	- 01000
m=2	- 01001
a=100	- 1

Decode the text "0101101101001"

Consider a file contents are encoded as **"0101101101001"**
using the following Huffman encoding

c=34	- 011
d=9	- 0101
g=35	- 00
u=2	- 01000
m=2	- 01001
a=100	- 1

Decoding the text

dacm

0101	1	011	01001
d	a	c	m